

Assignment Four

Coverage

By: Eng. Islam Hamdy Mohamed

Guided By: Eng. Hassan Khaled

1. Snippets for the implemented code

```
1 package enum_pkg;
2     typedef enum bit [1:0] {
3         No_Parity    = 2'b00,
4         ODD_Parity   = 2'b01,
5         EVEN_Parity  = 2'b10
6     } Parity_t;
7 endpackage
```

Figure 1 Enum Package Definition (enum_pkg.sv)

```
1 package Uart_packet;
2
3 import enum_pkg::*;
4
5 class Uart_class;
6     rand logic [7:0] data_in;
7     rand enum_pkg::Parity_t parity_in;
8
9     constraint Data_C {
10         data_in dist { 8'h00 :/ 30, 8'hFF :/ 30, [8'h01:8'hFE] :/ 40};
11         parity_in dist {No_Parity :/ 20, ODD_Parity :/ 40, EVEN_Parity :/ 40};
12     }
13
14     // --- FUNCTIONAL COVERAGE DEFINITION ---
15     covergroup uart_cg;
16         // Cover point for data input values (0x00, 0xFF, and middle values)
17         cp_data_in: coverpoint data_in {
18             bins zero = {8'h00};
19             bins max = {8'hFF};
20             bins mid = {[8'h01:8'hFE]};
21         }
22
23         // Cover point for parity modes
24         cp_parity_in: coverpoint parity_in {
25             bins no_p = {No_Parity};
26             bins odd_p = {ODD_Parity};
27             bins even_p = {EVEN_Parity};
28         }
29
30         // Cross coverage between Data and Parity modes
31         cross_data_parity: cross cp_data_in, cp_parity_in;
32     endgroup
33
34     function new();
35         uart_cg = new(); // Instantiate covergroup
36     endfunction
37
38     function void Print();
39         $display("displaying the stimulus values : data_in = %0h, parity_in = %0s",
40             data_in, parity_in.name());
41     endfunction
42
43     function Uart_class generate_stimulus();
44         Uart_class pkt = new();
45         if (pkt.randomize()) begin
46             pkt.uart_cg.sample(); // Sample coverage on successful randomization
47             pkt.Print();
48             return pkt;
49         end else begin
50             $display("Randomization failed");
51             return null;
52         end
53     endfunction
```

Figure 2 Uart_packet Class 1.1

```

1 task golden_model(ref logic expected_queue[$]);
2     expected_queue.delete();
3
4     expected_queue.push_back(1'b0);
5
6     for (int i = 0; i < 8; i++) begin
7         expected_queue.push_back(this.data_in[i]);
8     end
9
10    if (this.parity_in == enum_pkg::EVEN_Parity) begin
11        expected_queue.push_back(~(^this.data_in));
12    end else if (this.parity_in == enum_pkg::ODD_Parity) begin
13        expected_queue.push_back(^this.data_in);
14    end
15
16    expected_queue.push_back(1'b1);
17 endtask
18
19 task drive_stim(
20     ref bit clk,
21     ref bit tx_busy,
22     ref bit tx_start,
23     ref logic [7:0] tb_data_in,
24     ref bit parity_en,
25     ref bit even_parity
26 );
27     if (tx_busy) begin
28         $display("Notice: Transmitter is busy, waiting for it to finish...");
29         wait (tx_busy == 1'b0);
30     end
31
32     @(posedge clk);
33     tx_start    = 1'b1;
34     tb_data_in  = this.data_in;
35     parity_en   = (this.parity_in != enum_pkg::No_Parity);
36     even_parity = (this.parity_in == enum_pkg::EVEN_Parity);
37
38     @(posedge clk);
39     tx_start    = 1'b0;
40 endtask

```

Figure 3 Uart_packet Class 1.2

```

1  task collect_output(
2      ref bit clk,
3      ref bit tx_busy,
4      ref bit tx,
5      ref logic actual_queue[$]
6  );
7      int total_bits;
8      total_bits = (this.parity_in == enum_pkg::No_Parity) ? 10 : 11;
9
10     wait (tx_busy == 1'b1);
11     wait (tx == 1'b0);
12     repeat (total_bits) begin
13         @(negedge clk);
14         actual_queue.push_back(tx);
15     end
16     wait (tx_busy == 1'b0);
17 endtask
18
19 task check_result(ref logic actual_queue[$], ref logic expected_queue[$]);
20     bit match = 1;
21
22     if (actual_queue.size() != expected_queue.size()) begin
23         $display("[ERROR] Size mismatch! expected_size = %0d, actual_size = %0d",
24             expected_queue.size(), actual_queue.size());
25         match = 0;
26     end else begin
27         foreach (expected_queue[i]) begin
28             if (expected_queue[i] != actual_queue[i]) begin
29                 $display("[MISMATCH] Bit [%0d]: Expected = %0b, Actual = %0b",
30                     i, expected_queue[i], actual_queue[i]);
31                 match = 0;
32             end else begin
33                 $display("[MATCH] Bit [%0d]: Expected = %0b, Actual = %0b",
34                     i, expected_queue[i], actual_queue[i]);
35             end
36         end
37     end
38
39     if (match) begin
40         $display("[TEST PASSED] Frame transmitted correctly!\n");
41     end else begin
42         $display("[TEST FAILED] Mismatch detected in transmission!\n");
43     end
44
45     expected_queue.delete();
46     actual_queue.delete();
47 endtask
48
49 endclass
50 endpackage

```

Figure 4 Uart_packet Class 1.3

```

1  module Testbench;
2
3      import Uart_packet::*;
4      bit clk, rst_n, tx_start;
5      logic [7:0] data_in;
6      bit parity_en;    // 1 = enable parity
7      bit even_parity;  // 1 = even, 0 = odd
8      bit tx, tx_busy;
9      logic actual_queue[$], expected_queue[$];
10
11     // Clk generation
12     initial begin
13         clk = 0;
14         forever begin
15             #1 clk = ~clk;
16         end
17     end
18
19     // DUT connections
20     uart_tx DUT (
21         .clk(clk),
22         .data_in(data_in),
23         .rst_n(rst_n),
24         .tx(tx),
25         .tx_busy(tx_busy),
26         .tx_start(tx_start),
27         .parity_en(parity_en),
28         .even_parity(even_parity)
29     );
30
31     Uart_class pkt;

```

Figure 5 1.1 Top-Level Testbench Environment (Testbench.sv)

```

1  initial begin
2
3      // Reset all
4      rst_n = 0;
5      tx_start = 0;
6
7      // Check reset
8      @(negedge clk);
9      @(negedge clk);
10     @(negedge clk);
11     if (tx == 1'b1 && tx_busy == 1'b0) begin
12         $display("=====> Reset button work correctly <=====");
13     end else begin
14         $display("=====> Reset button does not work correctly <=====");
15     end
16
17     // Check transition from IDLE to Start
18     rst_n = 1'b1;
19     tx_start = 1'b1;
20     @(negedge clk);
21     tx_start = 1'b0; // Clear tx_start so DUT can complete and clear tx_busy!
22
23     if (tx == 1'b1 || tx_busy == 1'b1) begin
24         $display("Transition from IDLE to START state DONE");
25     end else begin
26         $display("Transition from IDLE to START state FAILED");
27     end
28
29     // Wait for DUT to return to IDLE before starting the stimulus loop
30     wait (tx_busy == 1'b0);
31
32     // Sending Data
33     repeat(100) begin
34         pkt = new();
35         pkt = pkt.generate_stimulus();
36
37         pkt.golden_model(expected_queue);
38         pkt.drive_stim(clk, tx_busy, tx_start, data_in, parity_en, even_parity);
39         pkt.collect_output(clk, tx_busy, tx, actual_queue);
40         pkt.check_result(actual_queue, expected_queue);
41     end
42
43     $display("=====> All Tests Completed <=====");
44     $stop();
45
46 end
47
48 endmodule

```

Figure 6 1.2 Top-Level Testbench Environment (Testbench.sv)

```

1  DATA: begin
2      tx <= shift_reg[0];           //FIX_ME::convert to 0 to fix ==> Done
3      shift_reg <= shift_reg >> 1; //FIX_ME::convert to >> to fix ==> Done
4      bit_cnt <= bit_cnt + 1;
5      if (bit_cnt == 4'd7) begin
6          if (parity_en)
7              state <= PARITY;
8          else
9              state <= STOP;
10     end
11 end
12

```

Figure 7 RTL Correction for Data Bit Order Inversion (DATA State)

➤ **Description:**

1. **Original Bug:** The design was initially transmitting data MSB-first by driving tx <= shift_reg[7] and shifting left (shift_reg << 1), which violates the standard LSB-first UART transmission protocol.
2. **Applied Fix:** Updated line 2 to transmit from bit 0 (tx <= shift_reg[0]) and line 3 to shift right (shift_reg <= shift_reg >> 1), ensuring correct LSB-to-MSB sequence.

```

1  IDLE: begin
2      tx <= 1'b1;
3      tx_busy <= 1'b0;
4      if (tx_start) begin
5          shift_reg <= data_in;
6          bit_cnt <= 4'd0;
7          parity_bit <= (even_parity) ? ~(^data_in) : (^data_in); // Islam :: First bug parity_bit <= (even_parity) ? ~(^data_in) : (^data_in); ==> Done
8          tx_busy <= 1'b1;
9          state <= START;
10     end
11 end
12

```

Figure 8 RTL Correction for Parity Bit Logic Inversion (IDLE State)

➤ **Description:**

1. **Original Bug:** The conditional ternary logic inside the IDLE state inverted the parity bit calculation—generating odd parity when even_parity = 1 and even parity when even_parity = 0.
2. **Applied Fix:** Corrected line 7 to parity_bit <= (even_parity) ? ~(^data_in) : (^data_in);, accurately generating Even parity for mode 1 and Odd parity for mode 0.

❖ Snippet for the waveform shows the different testcases

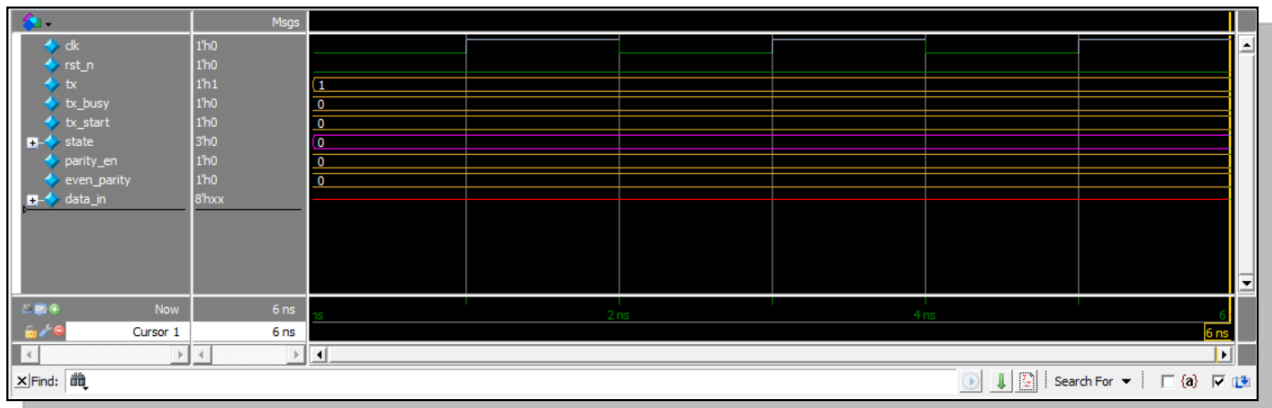


Figure 9 Waveform – Reset Assertion Verification (Active-Low Reset)

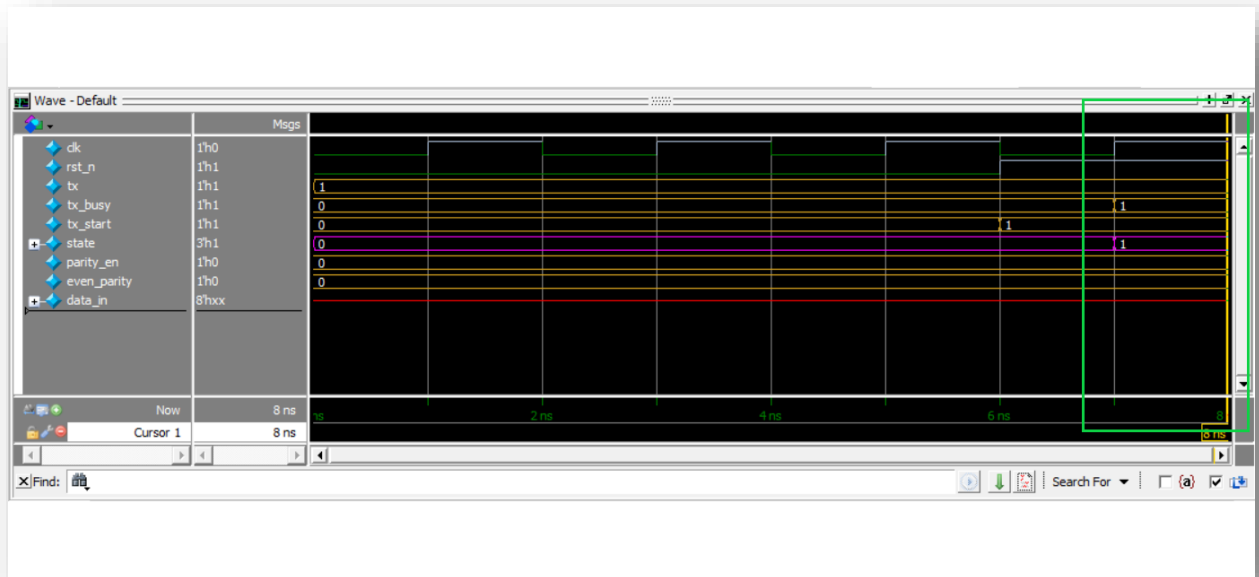


Figure 10 Waveform – FSM Transition from IDLE to START State

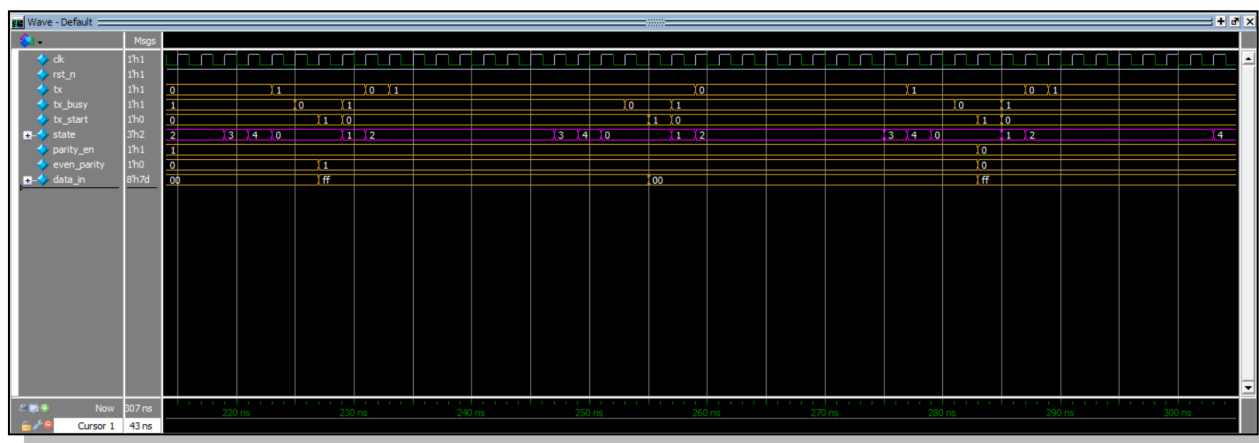


Figure 11 Waveform – Full FSM State Transitions Across Multiple Test Cases

- ❖ Snippets for the logs show the all printed values.

```
# =====> Reset button work correctly <=====
# ** Note: $stop      : Testbench.sv(40)
#      Time: 6 ns   Iteration: 1   Instance: /Testbench
```

Figure 12 Log(Check the reset button)

```
# Trantransition from IDLE to START state DONE
# ** Note: $stop      : Testbench.sv(51)
#      Time: 8 ns   Iteration: 1   Instance: /Testbench
```

Figure 13 Log(Transition From IDLE to Start)

```
displaying the stimulus values : data_in = 7d, parity_in = ODD_Parity
[MATCH] Bit [0]: Expected = 0, Actual = 0
[MISMATCH] Bit [1]: Expected = 1, Actual = 0
[MATCH] Bit [2]: Expected = 0, Actual = 0
[MATCH] Bit [3]: Expected = 1, Actual = 1
[MATCH] Bit [4]: Expected = 1, Actual = 1
[MATCH] Bit [5]: Expected = 1, Actual = 1
[MATCH] Bit [6]: Expected = 1, Actual = 1
[MATCH] Bit [7]: Expected = 1, Actual = 1
[MATCH] Bit [8]: Expected = 0, Actual = 0
[MISMATCH] Bit [9]: Expected = 0, Actual = 1
[MATCH] Bit [10]: Expected = 1, Actual = 1
[TEST FAILED] Mismatch detected in transmission!

displaying the stimulus values : data_in = f8, parity_in = ODD_Parity
[MATCH] Bit [0]: Expected = 0, Actual = 0
[MATCH] Bit [1]: Expected = 0, Actual = 0
[MISMATCH] Bit [2]: Expected = 0, Actual = 1
[MISMATCH] Bit [3]: Expected = 0, Actual = 1
[MATCH] Bit [4]: Expected = 1, Actual = 1
[MATCH] Bit [5]: Expected = 1, Actual = 1
[MATCH] Bit [6]: Expected = 1, Actual = 1
[MISMATCH] Bit [7]: Expected = 1, Actual = 0
[MISMATCH] Bit [8]: Expected = 1, Actual = 0
[MISMATCH] Bit [9]: Expected = 1, Actual = 0
[MISMATCH] Bit [10]: Expected = 1, Actual = 0
[TEST FAILED] Mismatch detected in transmission!
```

Figure 14 Log shows a mismatch in the transmission

```

# displaying the stimulus values : data_in = ff, parity_in = EVEN_Parity
# [MATCH] Bit [0]: Expected = 0, Actual = 0
# [MATCH] Bit [1]: Expected = 1, Actual = 1
# [MATCH] Bit [2]: Expected = 1, Actual = 1
# [MATCH] Bit [3]: Expected = 1, Actual = 1
# [MATCH] Bit [4]: Expected = 1, Actual = 1
# [MATCH] Bit [5]: Expected = 1, Actual = 1
# [MATCH] Bit [6]: Expected = 1, Actual = 1
# [MATCH] Bit [7]: Expected = 1, Actual = 1
# [MATCH] Bit [8]: Expected = 1, Actual = 1
# [MATCH] Bit [9]: Expected = 1, Actual = 1
# [MATCH] Bit [10]: Expected = 1, Actual = 1
# [TEST PASSED] Frame transmitted correctly!
#
# displaying the stimulus values : data_in = 0, parity_in = EVEN_Parity
# [MATCH] Bit [0]: Expected = 0, Actual = 0
# [MATCH] Bit [1]: Expected = 0, Actual = 0
# [MATCH] Bit [2]: Expected = 0, Actual = 0
# [MATCH] Bit [3]: Expected = 0, Actual = 0
# [MATCH] Bit [4]: Expected = 0, Actual = 0
# [MATCH] Bit [5]: Expected = 0, Actual = 0
# [MATCH] Bit [6]: Expected = 0, Actual = 0
# [MATCH] Bit [7]: Expected = 0, Actual = 0
# [MATCH] Bit [8]: Expected = 0, Actual = 0
# [MATCH] Bit [9]: Expected = 1, Actual = 1
# [MATCH] Bit [10]: Expected = 1, Actual = 1
# [TEST PASSED] Frame transmitted correctly!

```

Figure 15 Log for testing the corners

```

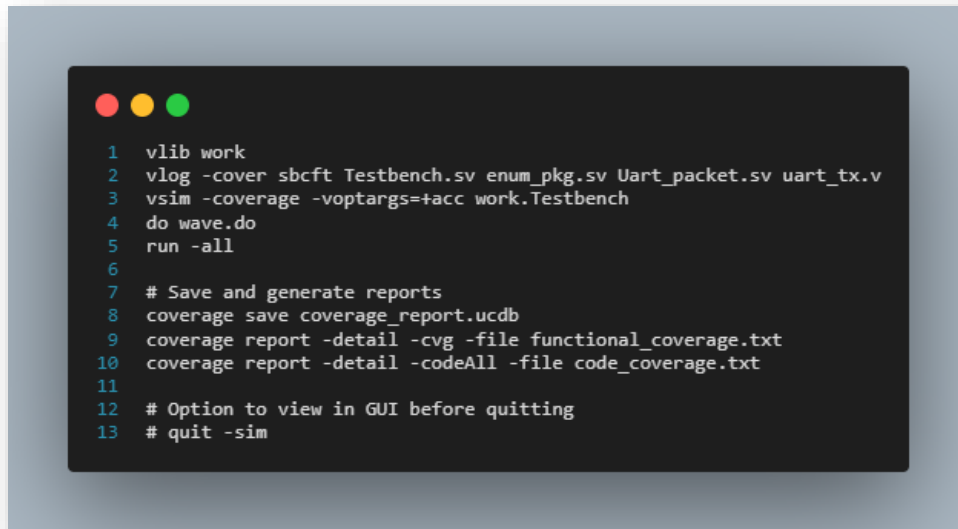
displaying the stimulus values : data_in = 7d, parity_in = ODD_Parity
[MATCH] Bit [0]: Expected = 0, Actual = 0
[MATCH] Bit [1]: Expected = 1, Actual = 1
[MATCH] Bit [2]: Expected = 0, Actual = 0
[MATCH] Bit [3]: Expected = 1, Actual = 1
[MATCH] Bit [4]: Expected = 1, Actual = 1
[MATCH] Bit [5]: Expected = 1, Actual = 1
[MATCH] Bit [6]: Expected = 1, Actual = 1
[MATCH] Bit [7]: Expected = 1, Actual = 1
[MATCH] Bit [8]: Expected = 0, Actual = 0
[MATCH] Bit [9]: Expected = 0, Actual = 0
[MATCH] Bit [10]: Expected = 1, Actual = 1
[TEST PASSED] Frame transmitted correctly!

displaying the stimulus values : data_in = f8, parity_in = ODD_Parity
[MATCH] Bit [0]: Expected = 0, Actual = 0
[MATCH] Bit [1]: Expected = 0, Actual = 0
[MATCH] Bit [2]: Expected = 0, Actual = 0
[MATCH] Bit [3]: Expected = 0, Actual = 0
[MATCH] Bit [4]: Expected = 1, Actual = 1
[MATCH] Bit [5]: Expected = 1, Actual = 1
[MATCH] Bit [6]: Expected = 1, Actual = 1
[MATCH] Bit [7]: Expected = 1, Actual = 1
[MATCH] Bit [8]: Expected = 1, Actual = 1
[MATCH] Bit [9]: Expected = 1, Actual = 1
[MATCH] Bit [10]: Expected = 1, Actual = 1
[TEST PASSED] Frame transmitted correctly!

```

Figure 16 Log for different data with different parity bits

❖ Snippet for Do file you have used for automating the process.



```
1 vlib work
2 vlog -cover sbcft Testbench.sv enum_pkg.sv Uart_packet.sv uart_tx.v
3 vsim -coverage -voptargs=+acc work.Testbench
4 do wave.do
5 run -all
6
7 # Save and generate reports
8 coverage save coverage_report.ucdb
9 coverage report -detail -cvg -file functional_coverage.txt
10 coverage report -detail -codeAll -file code_coverage.txt
11
12 # Option to view in GUI before quitting
13 # quit -sim
```

Figure 17 Do file used for automating the process

❖ Summary of Found Bugs & Fixes:

- Data Bit Order Inversion (MSB-First Transmission)
 - Issue: The original RTL transmitted data bits MSB-first (shift_reg[7] with shift_reg << 1), violating standard LSB-first UART protocol rules.
 - Resolution: Updated the DATA state logic to send LSB-first using tx <= shift_reg[0] and shift_reg <= shift_reg >> 1.
- Parity Bit Logic Inversion
 - Issue: The parity generation logic was inverted in the IDLE state, producing Odd parity when Even mode was selected and vice versa.
 - Resolution: Corrected the parity ternary statement to parity_bit <= (even_parity) ? ~(^data_in) : (^data_in).

✓ Code Coverage

```
1 =====Toggle Details=====
2
3 Toggle Coverage for File uart_tx.v --
4
5      Line      Node      1H->0L      0L->1H      "Coverage"
6 -----
7          3          rst_n          0          1          50.00
8
9 Total Node Count      =          31
10 Toggled Node Count   =          30
11 Untoggled Node Count =           1
12
13 Toggle Coverage      =          98.3% (61 of 62 bins)
14
15
16 Total Coverage By File (code coverage only, filtered view): 69.3%
```

Figure 18 End of code coverage report showing 98.3 % coverage

- Note : code_coverage.txt file was added to the .rar file

✓ Functional Coverage

```
1 COVERGROUP COVERAGE:
2 -----
3 Covergroup      Metric      Goal      Status
4 -----
5
6 TYPE /Uart_packet/Uart_class/uart_cg      100.0%      100      Covered
7 covered/total bins:      15      15
8 missing/total bins:      0      15
9 % Hit:      100.0%      100
10 Coverpoint uart_cg::cp_data_in      100.0%      100      Covered
11 covered/total bins:      3      3
12 missing/total bins:      0      3
13 % Hit:      100.0%      100
14 bin zero      21      1      Covered
15 bin max      34      1      Covered
16 bin mid      45      1      Covered
17 Coverpoint uart_cg::cp_parity_in      100.0%      100      Covered
18 covered/total bins:      3      3
19 missing/total bins:      0      3
20 % Hit:      100.0%      100
21 bin no_p      25      1      Covered
22 bin odd_p      37      1      Covered
23 bin even_p      38      1      Covered
24 Cross uart_cg::cross_data_parity      100.0%      100      Covered
25 covered/total bins:      9      9
26 missing/total bins:      0      9
27 % Hit:      100.0%      100
28 bin <zero,no_p>      6      1      Covered
29 bin <max,no_p>      9      1      Covered
30 bin <mid,no_p>      10      1      Covered
31 bin <zero,odd_p>      8      1      Covered
32 bin <max,odd_p>      11      1      Covered
33 bin <mid,odd_p>      18      1      Covered
34 bin <zero,even_p>      7      1      Covered
35 bin <max,even_p>      14      1      Covered
36 bin <mid,even_p>      17      1      Covered
37 CLASS Uart_class
38
39 TOTAL COVERGROUP COVERAGE: 100.0% COVERGROUP TYPES: 1
40
```

Figure 19 Functional Coverage report showing 100% coverage

- Note : functional_coverage.txt was added to the .rar file